

Simulation Design and Modeling Strategy

The Influence of Reviewer Expertise and Engagement on Peer Review of Grants

Sam Harper & Arijit Nandi

2026-08-19

Purpose of this document

This is a design and validation exercise for a project that aims to assess how reviewer engagement and expertise may affect grant scores during CIHR peer review. Given the restrictive nature of CIHR funding data, this document does not use real CIHR data; rather it lays out the data-generating process we believe matches CIHR's Project Grant peer review, the model we intend to fit against it, and evidence that the models we will use can actually recover simulated effects.

It is not a full pre-analysis plan (no pre-specified hypotheses, stopping rules, or multiplicity strategy yet), but it's close in spirit, and a subsequent revision could become one.

Research questions

Full detail is in the funded proposal (`cihr-ror-proposal.qmd`); briefly:

1. **Aim 1** – How do reviewer expertise (self-described: high/medium/low/not enough) and engagement (assigned reviewer vs. non-reviewing panelist) affect the impact of panel discussion on scores?
2. **Aim 2** – Do these effects differ by applicant characteristics, namely gender or career stage?
3. **Aim 3** (exploratory) – How would alternative funding-decision schemes (e.g. reweighting by engagement, partial randomization near the funding threshold) compare to the status quo?

The review process we're modeling

- The 3 assigned reviewers read the application and score it. At the meeting (and *before* any discussion), they are required to agree on a **consensus score**.
- After discussion all panel members (including the 3 reviewers) submit a **final score**. Reviewers are not bound to their own consensus number; they can move too.
- The final score must fall within ± 0.5 of the consensus score, and is entered to **one decimal place**.
- The (equally weighted) average of final scores across all panel members feeds the funding decision.

Data structure

We're assuming a three-level structure: committees, discussed applications nested in committees, and panel members nested in committees (crossed with applications, since a member reviews many applications within a cycle). Based on recent Project Grant committee sizes, our working numbers are **50 committees × 15 discussed applications per committee × 24 members per committee**.

CIHR’s Funding Analytics Team confirmed by email (2025-12-04) which fields are actually extractable for a data pull, versus fields that can only ever be touched by a CIHR analyst running our code in-house on the real data (full table and follow-up questions in `code/ror-research-log.qmd`). Headline for this document: committee/application/member identifiers, role (reviewer vs. panelist), self-described expertise, initial reviewer scores, consensus score, final scores, and funding result are all extractable. **Applicant gender and career stage – Aim 2’s entire basis – are not**, and won’t appear even in CIHR’s own distribution-matched dummy data. That constraint is why this document exists: the simulation below is the only rehearsal Aim 2’s code gets before it runs once, unsupervised, on real data.

Primary outcome: modeling the difference, not the level

The primary Aim 1 outlined above is to assess how *changes* from the initial consensus score to the final review scores (after discussion) may vary with panel expertise and engagement. Thus, rather than our primary outcome being the overall application score, we specifically want to model the difference between the consensus score and the final scores.

Let d_{ijk} be the final score minus the consensus score, for the i th panel member on the j th application in the k th committee:

$$d_{ijk} = \text{final score}_{ijk} - \text{consensus score}_{jk}$$

Our interest is in modeling d_{ijk} directly, rather than modeling the final score with consensus score as a covariate, for three reasons:

1. **Point mass at zero.** We hypothesize that most or many members will go with the consensus score, which leads to a mass either at the consensus (if you modeled the score) or a mass at zero if you model the difference. A member who doesn’t move from consensus has $d_{ijk} = 0$ regardless of which application they’re scoring. So even if you model the final score conditional on the consensus you will still have a spike, just relocated to wherever a given application’s consensus happened to land. Differencing standardizes the spike’s location (at zero) across every application and makes a shared two-part model feasible.
2. **Avoids some sources of confounding.** The consensus score is constant across everyone in the committee and reflects both the committee’s and the application’s “true quality” signal. Subtracting it off removes anything that’s shared by every member evaluating the same application (committee identity, application quality, etc.), leaving only the within-application variation across panel members as what’s left to explain. If some committees have both higher consensus scores and a different mix of reviewer expertise then consensus and ‘expertise’ are correlated across applications. Differencing eliminates this potential bias since the coefficient on consensus isn’t estimated at all.
3. **Matches the actual research question.** Aim 1 is about the *change from consensus* induced by discussion, not the *level* of the final score (driven mostly by application quality). Since it is simple to recover the final score like $\text{final score} = \text{consensus} + d_{ijk}$, anything Aim 3 needs at the score level for funding-decision simulations can be reconstructed downstream.

Estimand of interest

Our main quantity of interest in this project is the value that a given committee member’s score differential from the consensus (d_{ijk}) would take if a particular reviewer characteristic (role or experience) were set to a specific value, averaged over the entire population of panel members, applications, and

committees. We can write an example of, say, the difference between final scores for a given member-application pairing if that member were assigned to be a panelist or a reviewer:

$$\underbrace{\frac{1}{IJK} \sum_{k=1}^K \sum_{j=1}^J \sum_{i=1}^I}_{\text{Mean over members } i, \text{ applications } j, \text{ and committees } k}} \underbrace{(d_{ijk}(1) - d_{ijk}(0))}_{\text{Score if assigned to panelist(1) or reviewer(0)}}$$

Leaving aside for the moment the assumptions needed to credibly estimate this quantity, the specific data generating process for score deviations leads to challenges, since the distribution of this outcome depends on two processes: 1) whether a given member deviates from the consensus score; and 2) how large that deviation might be. Estimating these effects across the whole population of applications leads to the intuition for a two-part model.

Why a two-part model

We expect d_{ijk} to have a spike at exactly zero (members who simply adopt the consensus score) plus variation among those who depart from it. So the outcome is split into two linked models:

1. **Did this member deviate at all?** A Bernoulli/logistic model on $1[d_{ijk} \neq 0]$.
2. **If they deviated, by how much?** A model for the (signed) magnitude, conditional on deviating.

Estimating the expected value of the both parts across the entire population of reviewers (not just those that deviate from consensus) recovers the effect in the entire sample.

$$E[d_{ijk}] = \underbrace{P(\text{deviate})}_{\text{Part 1: any deviation at all}} \times \underbrace{E[\text{magnitude} \mid \text{deviate}]}_{\text{Part 2: size given deviation}}$$

This isn't just theoretically motivated – we checked it empirically before committing to the extra complexity. If most of the covariate signal about reviewer engagement/expertise actually lived in the *average size* of the deviation rather than *whether* a deviation happens, a single linear model on d_{ijk} would be simpler and would suffice.

```
d1 <- read_csv(here("data", "sim-data-aim1.csv"), show_col_types = FALSE)

m_linear <- lm(deviation ~ job + exp, data = d1)
m_logistic <- glm(deviated ~ job + exp, data = d1, family = binomial())

modelsummary(
  list("Linear: deviation" = m_linear, "Logistic: deviated (0/1)" = m_logistic),
  gof_omit = "DF|Deviance|R2|AIC|BIC|RMSE|Log.Lik|F",
  escape = TRUE
)
```

	Linear: deviation	Logistic: deviated (0/1)
(Intercept)	-0.003 (0.003)	-0.962 (0.059)
jobreviewer	-0.001	-0.345

	Linear: deviation	Logistic: deviated (0/1)
	(0.003)	(0.047)
explow	0.001	0.647
	(0.004)	(0.065)
expmed	0.003	0.520
	(0.004)	(0.067)
expnone	0.004	0.939
	(0.004)	(0.063)
Num.Obs.	18000	18000

The linear model’s coefficients on job/exp are tiny and mostly non-significant (largest around 0.005 points, on an outcome bounded at ± 0.5). The logistic model on the same covariates recovers the large, highly significant effect pattern actually built into the simulator (code/ror-sim-deviation.R). Because deviation magnitude is roughly symmetric around zero conditional on deviating, essentially all the covariate signal lives in *whether* someone deviates – a model that only looks at the conditional mean of d_{ijk} structurally cannot see it. This confirms the two-part structure is doing real work here, not adding complexity for its own sake.

Model specification (Aim 1)

Simulating the data-generating process

For the basic structure of the data generating process we use 50 committees, 15 discussed applications per committee, 24 members per committee. We also simulate another 15 applications that will end up being streamlined. For the assignments we have 3 of the 24 members on each application as the assigned reviewers, the rest are non-reviewing panelists. A consensus score is drawn per application (committee- and application-level random effects only, no member-level variation yet, since this is before any individual scoring happens). Whether each member deviates from that consensus is a function of their role and self-described expertise; if they deviate, the signed magnitude is drawn from a truncated distribution and rounded to the nearest tenth, matching CIHR’s one-decimal-place scoring (with rejection sampling so a “deviated” row can never round down to a contradictory zero).

```
# define parameters
cmte_n    = 50      # number of committees
app_n     = 15      # number of discussed applications per committee
mem_n     = 24      # number of committee members per committee

# candidate applications generated per committee before streamlining
app_n_candidates = app_n * 2

b0        = 4.1     # intercept for application's true underlying quality
u0c_sd    = 0.1     # random intercept SD for committee (quality level)
u0a_sd    = 0.3     # random intercept SD for application (quality level)

# signed magnitude of deviation, given deviation occurs, truncated to
# +/- 0.5 (CIHR's stated bound on final vs. consensus score)
```

```

dev_bias = 0      # placeholder: no systematic direction yet
dev_sd    = 0.15
dev_min   = -0.5
dev_max   = 0.5

# ... committee/application/member structure, reviewer assignment, and
# expertise assignment omitted here -- see code/ror-sim-aim1.R for
# the complete script

data <- data |>
  mutate(
    p_dev = plogis(a0 + (a1 * panelist) + (a2 * exp_high) +
      (a3 * exp_low) + (a4 * exp_none)),
    deviated = rbinom(n(), 1, p_dev),
    deviation = if_else(
      deviated == 1,
      round_tenth(rtruncnorm(n(), a = dev_min, b = dev_max,
        mean = dev_bias + u0m_bias, sd = dev_sd)),
      0)
  )

```

Every parameter above is at this point just an educated guess and a placeholder, since the point of the simulation is to see whether our modeling approach recovers a known effects, not predictive of what CIHR's actual numbers will look like.

Simulated data

What did we generate with the parameters above? Figure 1 shows the distribution of initial scores from the three reviewers, the consensus score and then a distribution of overall scores allowing for +/- 0.5 point deviations. Since the likely true consensus scores are not necessarily an average of the 3 reviewer scores (depending on calibration, etc.) we see a small excess in the left tail near 3.5. In practice a consensus score of 3.5 *among discussed applications* is likely more rare, but this seems a reasonable approximation.

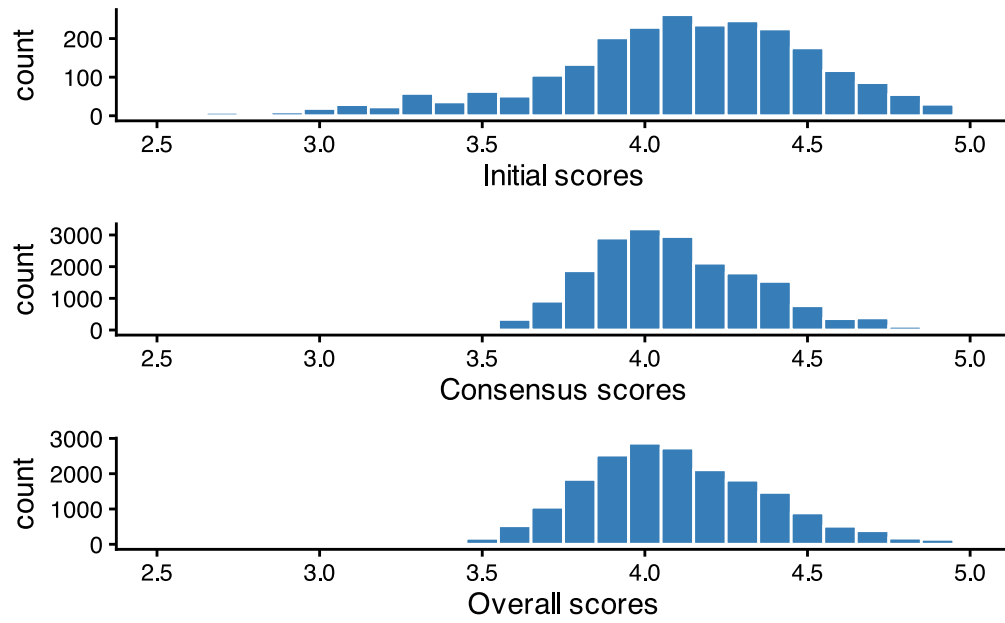


Figure 1: Distribution of simulated initial, consensus, and overall scores

Figure 2 also shows the odd distribution of deviations from the overall consensus score, with a large spike at zero (roughly 2/3rds of committee members going with the consensus):

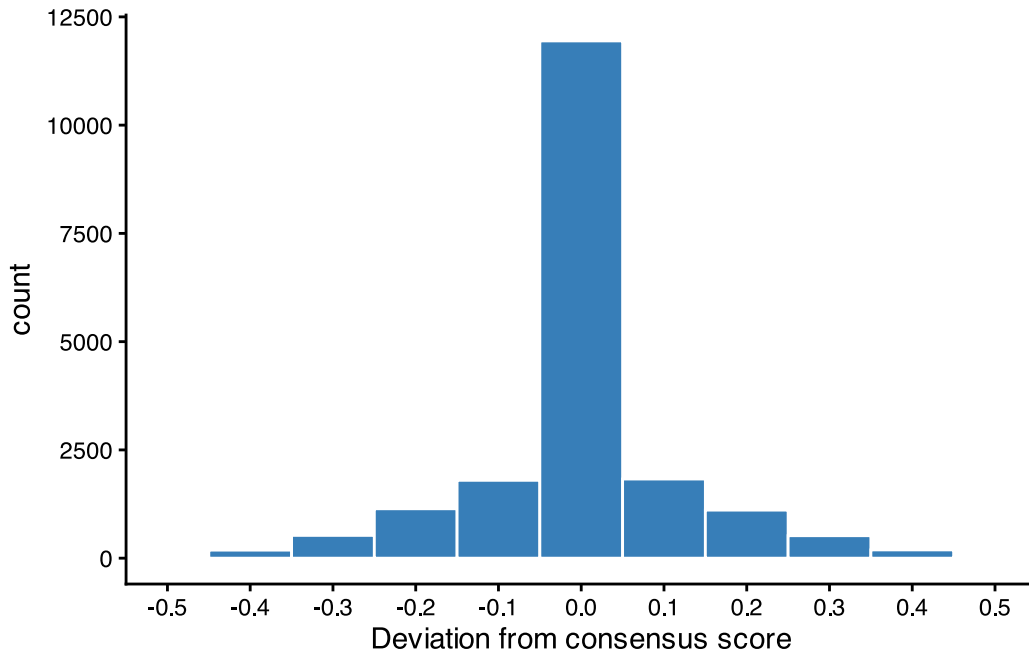


Figure 2: Distribution of deviations from overall consensus scores

Figure 3 shows how our initial simulation incorporates a small degree of variation across committees in the overall scores, as well as the average variation within committees across applications. For the application-level variation we average over committee by within-committee application rank.

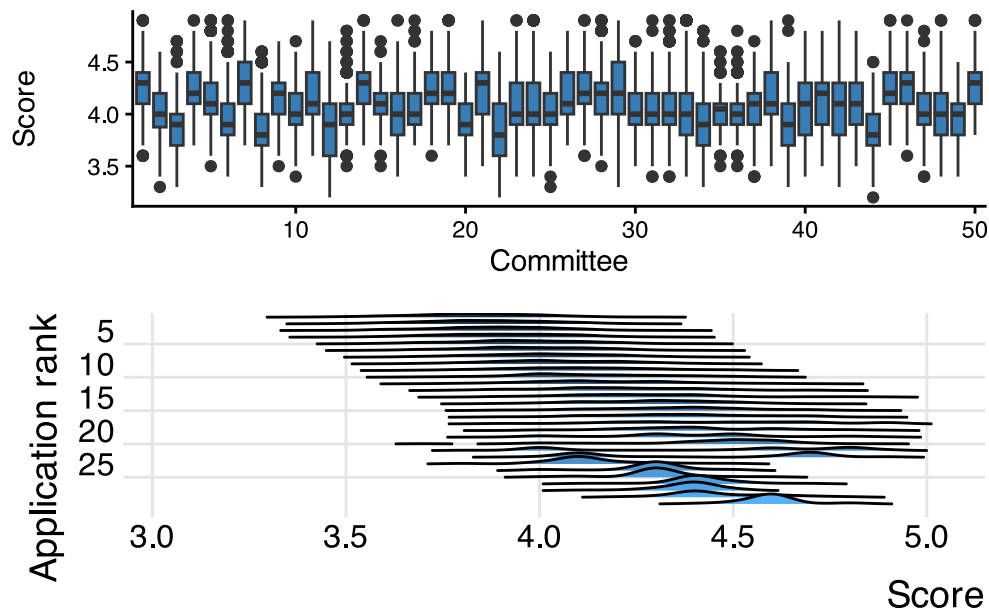


Figure 3: Distribution of score variation by committee and application

Part 1: did this member deviate?

```
ml_deviate <-
  brm(data = d1,
    family = bernoulli(),
    deviated ~ 1 + job + exp + (1 | cmte) + (1 | cid) + (1 | app),
    prior = c(prior(normal(0, 1.0), class = Intercept),
              prior(normal(0, 0.5), class = b),
              prior(exponential(1), class = sd)),
    iter = 2000, warmup = 1000, chains = 4, cores = 4,
    sample_prior = "yes",
    control = list(adapt_delta = 0.95))
```

Random intercepts for committee, member (cid), and application; fixed effects for role and expertise. Weakly informative priors throughout – normal(0, 1.5) on the (logit-scale) intercept, normal(0, 0.5) on coefficients, exponential(1) on group-level SDs. sample_prior = "yes" so we always have prior-predictive draws to check against before trusting the posterior.

This model run on the simulated data gives the following estimates, shown in Table 1:

Table 1

Parameter	Truth	Estimate	Error	95% CrI Lower	95% CrI Upper
<i>Fixed effects (log odds)</i>					
Intercept	-0.800	-0.773	0.047	-0.869	-0.679
Panelist vs. Reviewer	0.300	0.296	0.046	0.206	0.387
High vs. Medium Expertise	-0.400	-0.364	0.039	-0.438	-0.284
Low vs. Medium Expertise	0.200	0.210	0.040	0.127	0.293
None vs. Medium Expertise	0.500	0.461	0.042	0.378	0.546
<i>Random effects (SD)</i>					
Application		0.042	0.034	0.002	0.118
Committee Member		0.059	0.046	0.003	0.146
Committee		0.024	0.020	0.001	0.072

Estimates from deviation model

From the fixed effects we see that we generally recover the simulated parameters – the ‘true’ treatment effects in the first column are well approximated by our model. The three random-effect SDs are small and estimated with considerable uncertainty; the simulated data-generating process for whether a member deviates (deviated) depends only on job/exp, with no committee-, application-, or member-level heterogeneity built in at that stage (member-level heterogeneity only enters the *magnitude* of deviation, modeled separately in Part 2 below), so this is the expected pattern rather than a recovery failure. We can also generate the estimated absolute probabilities of deviating and how those are affected by job and expertise using the `marginaleffects` package

Part 2: how large, given a deviation?

CIHR scores are entered to one decimal place, so a deviator’s score can only land on one of 10 discrete steps ($\pm 0.1 \dots \pm 0.5$ relative to consensus). That’s genuinely ordinal/discrete data, not a continuous quantity with occasional extreme values – a continuous family (`student()`/`gaussian()`) would assign probability to impossible values and has no natural bound at ± 0.5 without truncation hacks. We use an ordinal `cumulative()` model instead, with flexible (non-equidistant) thresholds, since there’s no reason to assume the 10 steps are equally likely – and in the simulated data they clearly aren’t (concentrated near ± 0.1 , sparse at ± 0.5 , which happen to be the swings most relevant to Aim 3’s funding-decision question).

```
m1_magnitude <-
  brm(data = d1_dev,
    family = cumulative(link = "logit", threshold = "flexible"),
    deviation ~ 1 + job + exp + (1 | cmte) + (1 | cid) + (1 | app),
    prior = c(prior(normal(0, 1.5), class = Intercept),
              prior(normal(0, 0.5), class = b),
              prior(exponential(1), class = sd)),
    iter = 2000, warmup = 1000, chains = 4, cores = 4,
    sample_prior = "yes",
    control = list(adapt_delta = 0.95))
```

The cost: $E[\text{magnitude} \mid \text{deviate}]$ from an ordinal model is not a linear prediction – it’s a probability-weighted sum over the 10 category values, computed per posterior draw, not read off a default `marginalEffects` contrast. That combination step (and the corresponding $E[d_{ijk}] = P(\text{deviate}) \times E[\text{magnitude} \mid \text{deviate}]$ calculation across both models) is written but not yet implemented in code – see Open Questions below.

Software

Bayesian throughout, via `brms` + `cmdstanr` (Stan backend), matching the conventions in our other multilevel work. `marginalEffects` for posterior contrasts; models cached to `code/fits/` and deliberately re-fit (not silently reused) whenever formula or data change.

Extending to Aim 2: applicant characteristics

Since applicant gender and career stage will never be extractable – not even as CIHR’s own dummy data – this simulation is the only pre-deployment test that code touching those variables gets. We added them as **application-level** attributes (one value per application, like consensus score, not per member) to a new, separate simulation script (`code/ror-sim-aim2.R`) rather than growing the Aim 1 script further – see “A note on simulation architecture” below.

Critically, we didn’t just add *main* effects of gender/career stage. Aim 2’s actual question is whether the *engagement/expertise effect on deviation* differs by applicant characteristic – which requires an **interaction** term to exist in the generating process, or there’s nothing for the fitted model to recover.

```
# applicant-level main effects (modest -- not the effects of interest)
g1      = 0.1  # applicant gender = female
c1      = -0.1 # applicant career stage = early

# Aim 2's actual target: does the reviewer-engagement / expertise effect
# on P(deviate) differ by applicant gender / career stage? Baked in large
# on purpose so we can confirm the two-part model machinery actually
# recovers a real interaction, not just a real main effect.
i_panelist_female      = 0.5 # job(panelist) x gender(female)
i_exptime_earlycareer = 0.6 # exp(none) x career_stage(early)

data <- data |>
  mutate(
    p_dev = plogis(a0 + (a1 * panelist) + (a2 * exp_high) +
      (a3 * exp_low) + (a4 * exp_none) +
      (g1 * female) + (c1 * early_career) +
      (i_panelist_female * panelist * female) +
      (i_exptime_earlycareer * exp_none * early_career)),
    deviated = rbinom(n(), 1, p_dev)
  )
# ... deviation magnitude drawn the same way as Aim 1; applicant
# terms are deliberately left out of the magnitude part, consistent
# with the Aim 1 finding above that signal lives in whether someone
# deviates, not in the average size of the deviation
```

To be clear about what these numbers are and aren’t: they are chosen to be large enough to detect cleanly in a simulation of this size, not estimates – or even guesses – about the true magnitude

or even direction of any real effect in CIHR review. The goal is purely to confirm our statistical machinery would notice an interaction if the real data contained one.

```
d2 <- read_csv(here("data", "sim-aim2-data.csv"), show_col_types = FALSE)

m_aim2 <- glm(deviated ~ job * gender + exp * career_stage,
  data = d2, family = binomial())

coefs <- summary(m_aim2)$coefficients
coefs[grepl(":", rownames(coefs)), ] |>
  as_tibble(rownames = "term") |>
  tt(digits = 3)
```

term	Estimate	Std. Error	z value	Pr(> z)
jobreviewer:gendermale	0.571	0.0933	6.12	0.000000000933
explore:career_stageestablished	0.26	0.1228	2.12	0.034154693615
expmed:career_stageestablished	0.253	0.1281	1.97	0.048458324937
expnone:career_stageestablished	-0.43	0.1195	-3.6	0.00032331474

Both interactions are recovered cleanly and with high significance (the sign flips relative to the parameters above are just R’s alphabetical choice of reference category – job’s reference is “panelist”, gender’s is “female”, career_stage’s is “early”, so e.g. jobreviewer:gendermale is algebraically the same interaction as job(panelist):gender(female), just from the opposite corner of the 2x2).

Not yet built: an Aim-2-specific version of code/ror-analysis-score-models.R that actually includes these interaction terms in the brm() formulas. The simulation is ready; the model script that would be tested against it isn’t written yet.

Aim 3: not yet designed

Aim 3 asks how alternative funding-decision schemes – reweighting scores by engagement, or partially randomizing funding decisions for applications near the threshold – would compare to the status quo. That’s a structurally different kind of simulation: an intervention/counterfactual layered on top of the funding decision itself, not just an extension of Aims 1-2’s data-generating process for reviewer behavior. We haven’t started designing it, and would welcome input on what the alternative schemes worth simulating actually are before building anything.

A note on simulation architecture

We’re deliberately keeping a separate, self-contained simulation script per aim (ror-sim-deviation.R for Aim 1, ror-sim-aim2.R for Aim 2, and eventually an Aim 3 script) rather than one script that keeps growing. Some duplication of the committee/application/member setup across scripts is an accepted cost of each one being independently runnable and easy to reason about in isolation.

What this simulation validates, and what it doesn’t

It validates that: (1) the two-part model structure is necessary, not over-engineering, for this outcome; (2) the ordinal magnitude submodel is the right match for CIHR’s one-decimal-place scoring; (3) the model can recover both main effects (Aim 1) and interaction effects (Aim 2) of a detectable size, using

only the fields CIHR has confirmed are extractable plus the fields that would only ever be touched via an in-house CIHR-run analysis.

It does **not** tell us anything about the true size, direction, or even existence of any of these effects in actual CIHR peer review – every parameter in both simulations is an illustrative placeholder chosen for statistical detectability, not drawn from evidence.

Open questions

For Arijit

- Is d_{ijk} (final – consensus) the right primary outcome, or is there a better way to frame the estimand for Aim 1/2?
- Does the interaction specification for Aim 2 (job x gender, exp x career_stage, both entering only the deviation-probability part) match how you’d operationalize the research question, or would you frame it differently?
- Any concerns about the priors, the random-effects structure, or the choice of an ordinal rather than continuous family for the magnitude part?
- Anything about the overall statistical strategy that looks wrong before this goes to CIHR?

For CIHR’s Funding Analytics Team

(Carried over from `code/ror-research-log.qmd`; repeated here since this document is the more likely vehicle for actually sending them.)

1. For fields that can’t be extracted (applicant gender/career stage, reviewer gender/experience, etc.) – can the dummy-data step still include *synthetic* versions of those columns for us to pipeline-test against, or is the in-house-run path a black box until the real run?
2. Resubmission status is only available from 2023 onward – does that mean our cohort has to start at the 2023 competition if we want that field, or can it be requested as a partially-missing covariate over a longer window?
3. How should we define/measure “reviewer experience” for the fields where that’s the blocker to a yes/no on extractability?
4. Can you confirm our understanding of the review process itself (the ± 0.5 rule, one-decimal-place scoring, equal-weight averaging into the funding decision) is accurate?
5. How much variation exists in the number of members across committees, applications per committee, and scores per application (given conflicts)?
6. Discussion time? Do we know how long the discussion period goes at the application level?

What comes next

Once the Aim 2 model script exists and `FIT_MODELS` is switched on – after confirming CIHR’s execution environment can actually compile/run Stan via `cmdstanr`, and after prior-predictive checks on both submodels – this document (or its next revision) will add a results section presenting posterior marginal effects via `marginalEffects` for both the deviation-probability and magnitude submodels, and showing recovery of the known simulated effects at the full posterior level, the same way the frequentist checks above do at a point-estimate level.

Appendix: Aim 1 simulation script (ror-sim-deviate.R)

The “Model specification” section above walks through a simplified, didactic version of the Aim 1 data-generating process. `code/ror-sim-deviate.R` is the actual, current simulation script – the one that produced the results reported in that section and the workhorse we’re using going forward – and it’s more procedurally involved than the summary above lets on. Rather than re-typing or excerpting pieces of it here (a second, hand-maintained copy that can silently drift out of sync with the real script, a failure mode we’ve already hit once with the simulation’s own file names), this appendix pulls in the script’s full, current contents directly, so it can never go stale. What follows is a guide to what to look for as you read it.

The script is organized into seven numbered sections, matching the `## N ...` comments in the source:

- **0-1: packages and parameters.** All parameters are illustrative placeholders, not estimates – with two exceptions called out explicitly in the script’s own comments: `streamline_rank_threshold = 0.60` (CIHR’s stated real streamlining rule) and the beta-distributed committee pool size range, `[20, 80]` (CIHR’s own stated range). `init_sd_lo` and the `exp` category mix were calibrated against real reviewer-score data Sam has access to, though that source data itself isn’t reproduced or logged anywhere in this repository.
- **2: candidate pool.** Generates each committee’s full pool of candidate applications (before streamlining), assigns 3 of each application’s 24 committee members as reviewers, draws each reviewer’s initial score, and has each reviewer separately record a “top”/“bottom” call – correlated with, but not determined by, their score. Out-of-bounds initial scores are redrawn rather than clamped, to avoid an artificial probability spike at the scale’s boundaries. The application grid is built manually (not via `faux::add_random()`) specifically to avoid a conflation bug found earlier in this project: `add_random()`’s crossed factors give identical random-effect draws to same-labeled applications across different committees unless a unique composite key (`aid`) is used instead.
- **3: streamlining decision.** Implements CIHR’s actual rule as Sam described it from his own committee/Scientific Officer experience: an application is streamlined out (not discussed) if at least one reviewer called it “bottom” *and* its mean-of-3 score ranks in the bottom 60% of that committee’s own candidate pool – a relative, within-committee rule, not a fixed absolute score cutoff. This mechanism was specifically validated against two hypothetical datasets Sam constructed, including a case where two applications with identical reviewer scores had different discussed/not-discussed outcomes – something a pure score-threshold rule can’t produce, but this rank-and-call combination can. An earlier “bring back” stage, where a reviewer could argue to reinstate a borderline streamlined-out application, was tried and deliberately dropped as too haphazard and too thinly grounded (only ~6% of discussed applications) to be worth the added complexity – see `code/ror-research-log.qmd` for that discussion.
- **4: the two-part deviation model.** Once discussed, each member’s score is generated as consensus plus a two-part process: whether they deviate from consensus at all (a function of role and self-described expertise), and, if so, the signed magnitude (a truncated, rounded draw, with member-level leniency/harshness heterogeneity, `u0m_bias`). This is the same two-part structure the modeling strategy above is built to recover.
- **5: internal consistency checks.** A set of `stopifnot()` assertions – e.g., every committee retains at least one discussed application, consensus never falls outside the range of its own 3 reviewer scores, deviation is exactly zero whenever `deviated == 0` and never zero when `deviated == 1`. These

aren't diagnostics for a reader; they're guardrails that halt the script if a future edit breaks an invariant the rest of the design depends on.

- **6: empirical checks.** Printed summaries – pool size and discussion-rate distributions, confirmation that the identical-scores-different-outcomes pattern the mechanism was designed to explain actually occurs, and a quick `glm()/lmer()` check that job/exp effects on deviation and member-level heterogeneity both still recover cleanly despite the richer selection stage.
- **7: output.** Writes `data/sim-deviate.csv`, which `code/ror-analysis-score-models.R` reads.

```
# program:  ror-sim-deviate.R
# task:     generating a more faithful streamlining DGP
#           for Aim 1
# input:    none (simulated from scratch)
# output:   data/sim-deviate.csv
# project:  RoR
# author:   sam harper \ 2026-08-19
#
# note:     1. each of the 3 assigned reviewers gives a score AND a
#           separate categorical "top" (competitive) / "bottom"
#           (not competitive) call -- correlated with their score,
#           but not a deterministic function of it.
#           2. streamlining rule: an application is streamlined out
#           (not discussed) iff >=1 reviewer called it "bottom"
#           AND its mean-of-3 score ranks in the bottom 60% of
#           *that committee's own* candidate pool (a relative/rank
#           rule, not a fixed absolute score).
#
#           This exactly explains a pattern found in a hypothetical
#           dataset Sam constructed earlier the same day: two
#           applications with *identical* 3 reviewer scores had
#           different discussed/not-discussed outcomes, which a pure
#           score-threshold rule cannot produce but this mechanism can
#           (different top/bottom calls and/or different committee
#           rank context at the same raw score). A "bring back"
#           advocacy stage was tried on top of this and deliberately
#           dropped -- see the note where it used to live, in section 3
#           below, and ror-research-log.qmd.
#
#           All streamlining-specific parameters below (tb_center,
#           tb_slope) are illustrative placeholders invented to make
#           the mechanism behave sensibly -- same status as every other
#           parameter in this project's simulations, not estimates.
#           streamline_rank_threshold = 0.60 is a stated real rule, not
#           a guess.
#
#           Committee-level application-pool size (2026-08-14): was a
#           fixed pool_per_cmte = 40 for every committee; now drawn per
#           committee from a beta distribution on CIHR's own stated
#           [20, 80] range (cihr-irsc.gc.ca/e/51315.html), instead of a
#           real basis that didn't exist when 2026-08-06 explicitly
#           decided to leave committee size fixed. The beta's shape
#           parameters are illustrative -- chosen to roughly match the
#           range/right-skew implied by a one-off back-calculation from
#           real CIHR funded-application counts (see
#           code/ror-cihr-committee-size-check.R and
```

```

#           ror-research-log.qmd), not fit to it or drawn from it at
#           runtime -- deliberately not making this script depend on
#           that dataset for anything beyond a rough ballpark.

## 0 Load needed packages ----
library(here)
library(tidyverse)
library(faux)
library(truncnorm)

set.seed(20260819)

## 1 Define parameters ----

cmte_n = 50      # number of committees
mem_n  = 24      # number of committee members per committee

# Per-committee pool_size is drawn (below, section 2) from a beta
# distribution scaled to CIHR's own stated 20-80 range.

pool_min = 20      # lower bound on applications reviewed per committee
pool_max = 80      # upper bound
pool_shape1 = 2.5  # rbeta() shape -- illustrative, not fit; chosen to
pool_shape2 = 4      # roughly match the mean/SD/right-skew of the
                    # funded-count back-calculation (mean ~42, SD ~12)

b0        = 4.0    # intercept for (discussed) application's true quality
u0c_sd    = 0.1    # random intercept SD for committee (quality level)
u0a_sd    = 0.3    # random intercept SD for application (quality level)

# longer tail for lower-ranked applications. init_sd_lo raised from 0.30
# to 1.1 (2026-08-14): among DISCUSSED applications specifically, the
# any_bottom-AND-rank streamlining rule (section 3) means a reviewer's
# very low individual score can only survive to discussion if the
# other 2 reviewers' scores are high enough to keep the consensus above
# the committee's 60th-percentile rank -- an inherently rare
# combination. At the original 0.30, that combination essentially never
# occurred (0 discussed reviewer-scores below 3.4), which was too tight
# given real reviewer-score data Sam has access to, where a meaningful
# share of discussed applications carry within-application reviewer
# disagreement of a point or more, and individual scores as low as the
# low 2s. Tried a small-probability "discordant outlier" mixture first,
# but even at implausibly extreme settings (30% of reviews, SD 1.3) it
# barely moved the count -- the real lever is just the overall low-tail
# SD, which lengthens the tail with only a modest effect on the bulk of
# the distribution. Calibrated to roughly match that real reference
# distribution's share of discussed reviewer-scores below 3.5 (~8%) and
# below 3.0 (~1.5%); see ror-research-log.qmd.
init_sd_lo = 1.1
init_sd_hi = 0.15 # reviewer-noise SD above it (tighter)

# probability of deviation from consensus (logit scale), and the
# signed magnitude given deviation
# this is the "hurdle"-equivalent part

```

```

a0      = -0.8  # baseline logit prob. of deviating
a1      =  0.3  # panelist vs. reviewer
a2      = -0.4  # high expertise
a3      =  0.2  # low expertise
a4      =  0.5  # no expertise

# signed magnitude of deviation, given deviation occurs, truncated to
# +/- 0.5 (CIHR's stated bound on final vs. consensus score)
dev_bias = 0      # population-average bias: still none (see u0m_bias_sd
# below for between-member variation around this)
dev_sd   = 0.15
dev_min  = -0.5
dev_max  = 0.5

# between-member SD in habitual leniency/harshness -- some members
# consistently deviate a bit high, some a bit low, across every
# application they review, but the population average stays at
# dev_bias (0). Not a fixed direction for everyone (that would just be
# dev_bias != 0); this is heterogeneity *across* members. One draw per
# unique cid, not per raw member-slot.
u0m_bias_sd = 0.1

scale_min = 0      # lower bound of the scoring scale
score_max = 4.9    # upper bound of the scoring scale

round_tenth <- function(x) round(x * 10) / 10

# -- streamlining mechanism parameters --

# each reviewer's "top"/"bottom" call: p(bottom) = plogis(tb_slope *
# (tb_center - score)) -- tb_center is roughly where a reviewer is as
# likely to say "top" as "bottom"; tb_slope controls how tightly the
# call tracks their own score (higher = closer to deterministic)
tb_center = 3.9
tb_slope  = 6

# initial rule: streamlined out iff >=1 "bottom" call AND rank (of the
# mean of 3 scores, within this committee's own candidate pool) is at
# or below this percentile. This 0.60 is the one number here that's a
# stated real rule, not an invented placeholder.
streamline_rank_threshold = 0.60

## 2 Stage 1: candidate pool -- reviewer scores and top/bottom calls ----
## Mirrors ror-sim-aim1.R's committee/application/member setup and
## reviewer-score generation exactly (including the redraw-not-clamp
## handling of out-of-bounds scores); the streamlining logic that
## follows in section 3, and the variable per-committee pool_size below,
## are new.
##
## The grid is built manually (reframe + cross_join) rather than via
## add_random(application = ...), because add_random()'s "application"
## factor would be fully crossed with committee (same pool_per_cmte
## levels recycled identically in every committee) even when pool sizes
## vary by committee -- and add_ranef() on that raw factor would give

```

```

## identical u0a draws to same-labeled applications across different
## committees. This is the same conflation bug found and fixed in
## ror-sim-aim1.R/ror-sim-aim2.R on 2026-08-14 (there for "member";
## fixed here by never creating the crossed factor in the first place
## for "application"), via an explicit unique `aid` composite key.

cmte_pool <- tibble(
  cmte = sprintf("%02d", 1:cmte_n),
  pool_size = round(pool_min +
    (pool_max - pool_min) * rbeta(cmte_n, pool_shape1, pool_shape2))
)

cmte_app <- cmte_pool |>
  reframe(app = seq_len(pool_size), .by = c(cmte, pool_size)) |>
  select(cmte, app)

data <- cmte_app |>
  cross_join(tibble(memno = sprintf("%02d", 1:mem_n))) |>
  mutate(
    cid = paste0(cmte, "_", memno),
    aid = paste0(cmte, "_", app)
  ) |>

  add_ranef("cmte", u0c = u0c_sd) |>
  add_ranef("aid", u0a = u0a_sd) |>

  group_by(cmte, app) |>
  mutate(
    job = sample(c(rep("reviewer", 3),
      rep("panelist", mem_n - 3))),
    exp = sample(c(rep("high", 6),
      rep("med", 10), rep("low", 4),
      rep("none", 4))),
    z_init = rnorm(n()),
    init_score = if_else(job == "reviewer",
      round_tenth(b0 + u0c + u0a +
        if_else(z_init < 0, z_init * init_sd_lo, z_init * init_sd_hi)),
      NA_real_)
  ) |>
  ungroup() |>
  select(-z_init)

# redraw any reviewer's init_score that landed outside the scale's true
# bounds, rather than clamping it -- identical mechanism to
# ror-sim-aim1.R
out_idx <- which(!is.na(data$init_score) &
  (data$init_score < scale_min | data$init_score > score_max))
while (length(out_idx) > 0) {
  z <- rnorm(length(out_idx))
  noise <- if_else(z < 0, z * init_sd_lo, z * init_sd_hi)
  data$init_score[out_idx] <- round_tenth(
    b0 + data$u0c[out_idx] + data$u0a[out_idx] + noise)
  out_idx <- which(!is.na(data$init_score) &
    (data$init_score < scale_min | data$init_score > score_max))
}

```

```

}

# each reviewer's separate top/bottom call, based on their now-finalized
# init_score -- correlated with score, not determined by it
data <- data |>
  mutate(
    p_bottom = plogis(tb_slope * (tb_center - init_score)),
    # rbinom() on the NA p_bottom values (panelist rows) warns even
    # though the result is discarded by the outer if_else() below --
    # substitute a dummy 0 there rather than let it warn every run
    top_bottom = if_else(job == "reviewer",
      if_else(rbinom(n(), 1, if_else(is.na(p_bottom), 0, p_bottom)) == 1,
        "bottom", "top"),
      NA_character_)
  ) |>
  select(-p_bottom)

## 3 Streamlining decision ----

# application-level summaries, broadcast back to every member-row
data <- data |>
  group_by(cmte, app) |>
  mutate(
    consensus = round_tenth(mean(init_score, na.rm = TRUE)),
    any_bottom = any(top_bottom == "bottom", na.rm = TRUE)
  ) |>
  ungroup()

# decide streamlining once per application (on distinct app-level rows,
# not the 24x-duplicated member-level rows -- rank has to be computed
# against other *applications*, not other rows), then join back
decision <- data |>
  distinct(cmte, app, consensus, any_bottom) |>
  group_by(cmte) |>
  mutate(rank_pct = percent_rank(consensus)) |>
  ungroup() |>
  mutate(
    discussed = !(any_bottom & (rank_pct <= streamline_rank_threshold))
  ) |>
  select(cmte, app, consensus, rank_pct, discussed)

stopifnot(
  "every committee should have at least 1 discussed application" =
    data |> left_join(decision |> select(-consensus), by = c("cmte", "app")) |>
      filter(discussed) |> distinct(cmte, app) |>
        count(cmte) |> pull(n) |> min() >= 1
)

data <- data |>
  left_join(decision |> select(-consensus), by = c("cmte", "app")) |>
  filter(discussed) |>

  mutate(
    panelist = if_else(job == "panelist", 1, 0),

```

```

exp_high = if_else(exp == "high", 1, 0),
exp_low  = if_else(exp == "low", 1, 0),
exp_none = if_else(exp == "none", 1, 0)
) |>

# member-level leniency/harshness trait
# add_ranef on cid, not faux's crossed "member" factor
add_ranef("cid", u0m_bias = u0m_bias_sd)

## 4 Two-part deviation from consensus ----
## CIHR scores (both consensus and individual final scores) can only be
## entered to one decimal place, so deviation -- and hence score -- must
## also land on a tenth. A raw truncnorm() draw is continuous, so we round
## it and then redraw any deviated == 1 case that rounds to exactly 0 (a
## "deviator" can't end up with a final score identical to consensus).

data <- data |>
mutate(
  p_dev = plogis(a0 + (a1 * panelist) + (a2 * exp_high) +
    (a3 * exp_low) + (a4 * exp_none)),
  deviated = rbinom(n(), 1, p_dev),
  deviation = if_else(
    deviated == 1,
    round_tenth(rtruncnorm(n(), a = dev_min, b = dev_max,
      mean = dev_bias + u0m_bias, sd = dev_sd))),
    0)
)

zero_idx <- which(data$deviated == 1 & data$deviation == 0)
while (length(zero_idx) > 0) {
  data$deviation[zero_idx] <- round_tenth(
    rtruncnorm(length(zero_idx), a = dev_min, b = dev_max,
      mean = dev_bias + data$u0m_bias[zero_idx], sd = dev_sd))
  zero_idx <- which(data$deviated == 1 & data$deviation == 0)
}

data <- data |>
mutate(
  score = round_tenth(pmax(scale_min,
    pmin(score_max, consensus + deviation)))
) |>
select(-u0c, -u0a, -u0m_bias, -p_dev)

## 5 Checks ----

stopifnot(
  "row count should be a whole number of mem_n-sized application blocks" =
    nrow(data) %% mem_n == 0,
  "expect exactly 3 reviewers per committee-application" =
    data |> filter(job == "reviewer") |>
    count(cmte, app) |> pull(n) |> unique() == 3,
  "expect exactly 24 members per committee-application" =
    data |> count(cmte, app) |> pull(n) |> unique() == mem_n,
  "expect exactly 3 non-missing initial reviewer scores per application" =

```

```

data |> filter(!is.na(init_score)) |>
  count(cmte, app) |> pull(n) |> unique() == 3,
"consensus should never fall outside the range of the 3 reviewers' own initial scores" =
data |> group_by(cmte, app) |>
  summarise(
    lo = min(init_score, na.rm = TRUE),
    hi = max(init_score, na.rm = TRUE),
    cons = first(consensus), .groups = "drop") |>
  summarise(ok = all(cons >= lo & cons <= hi)) |> pull(ok),
"every retained application should actually be flagged discussed" =
  all(data$discussed),
"deviated should be 0/1" =
  all(data$deviated %in% c(0, 1)),
"deviation should be exactly 0 when deviated == 0" =
  all(data$deviation[data$deviated == 0] == 0),
"score should stay within [scale_min, scale_max]" =
  all(data$score >= scale_min & data$score <= scale_max),
"deviated == 1 rows should never have a zero deviation after rounding" =
  all(data$deviation[data$deviated == 1] != 0)
)

## 6 Empirical checks ----

n_candidates_total <- sum(cmte_pool$pool_size)
n_discussed_total <- data |> distinct(cmte, app) |> nrow()

cat("=== per-committee pool size (drawn from beta on CIHR's stated [20,80] range) ===\n")
print(summary(cmte_pool$pool_size))
cat("SD across committees:", round(sd(cmte_pool$pool_size), 2), "\n\n")

cat("=== streamlining mechanism ===\n")
cat("overall discussion rate:",
  round(n_discussed_total / n_candidates_total, 3), "\n")

per_cmte <- data |> distinct(cmte, app) |> count(cmte, name = "n_discussed")
cat("discussed applications per committee -- summary:\n")
print(summary(per_cmte$n_discussed))
cat("SD across committees:", round(sd(per_cmte$n_discussed), 2), "\n\n")

cat("=== can identical consensus scores still produce different outcomes? ===\n")
## uses `decision`, which covers the FULL candidate pool (both discussed
## and streamlined-out) -- `data` by this point only contains the
## discussed survivors, so checking against `data` alone could never
## find a not-discussed match
dupe_check <- decision |>
  group_by(consensus) |>
  filter(n_distinct(discussed) > 1) |>
  ungroup()
cat("distinct consensus values with both outcomes present:",
  n_distinct(dupe_check$consensus), "\n")
dupe_check |> arrange(consensus) |>
  select(cmte, app, consensus, rank_pct, discussed) |>
  slice_head(n = 8) |> print()
cat("\n")

```

```

cat("=== does job/exp on P(deviate) still recover cleanly despite the richer selection
stage? ===\n")
m_check <- glm(deviated ~ job + exp, data = data, family = binomial())
print(summary(m_check)$coefficients)

cat("\n=== member-level heterogeneity still recoverable? ===\n")
suppressMessages(library(lme4))
m_member_check <- lmer(deviation ~ 1 + (1 | cid),
  data = data |> filter(deviated == 1))
print(VarCorr(m_member_check))

## 7 Write output ----

write_csv(data, here("data", "sim-deviate.csv"))

```

References